A D PATEL INSTITUTE OF TECHNOLOGY
(The Charutar Vidya Mandal University)
GSIRE An ISO 9001:2015 Certified Institute

Smart Route Finder: Shortest Path Finding Using Dijkstra's Algorithm

A Project report submitted
In partial fulfilment of the requirements for the Degree of
Bachelor of Technology

In
Information Technology
Semester – V
Subject: Design and Analysis of Algorithms

By

Savani Ayushi Jivrajbhai (12402080601014)

Faculty Guide
Minakshi Patil
Assistant Professor

Academic Year 2026-27 (Odd)

Department of Information Technology
A D Patel Institute of Technology
New Vallabh Vidyanagar
Vitthal Udyognagar - 388121
Anand (Gujarat), India

CERTIFICATE

This is to certify that the project work embodied in this report entitled, "**Smart Route Finder: Shortest Path Finding Using Dijkstra's Algorithm**" was carried out by **Savani Ayushi Jivrajbhai (12402080601014)** at A D Patel Institute of Technology for partial fulfilment of B.Tech. degree to be awarded by the Charutar Vidya Mandal (CVM) University, in the subject Design and Analysis of Algorithms. This project work has been carried out under my supervision and is to the satisfaction of the department.

Date:

Place:

Guide
Minakshi Patil
Assistant Professor

Head of Department
Dr. Priyang Bhatt
Professor & Head

Acknowledgement

I would like to express my sincere thanks to my respected faculty guide for guiding me throughout the preparation of this Smart Route Finder project and report. Their support, advice and encouragement helped me to understand the Design and Analysis of Algorithms subject better and to complete my work successfully.

I am also thankful to A D Patel Institute of Technology and the Department of Information Technology for providing me with the opportunity and the resources to complete this project. The learning environment and the lab facilities made it easier to build and test the application.

I would like to thank my parents for their constant support and motivation. Their encouragement helped me stay focused and complete this work on time.

I am also grateful to my friends and classmates who helped me by sharing ideas and supporting me whenever needed.

Lastly, I would like to thank all those who directly or indirectly helped me in completing this project. It has been a great learning experience for me.

Savani Ayushi Jivrajbhai
(12402080601014)

Abstract

Finding the shortest route between two places is one of the most common problems in computer science. A road network can have many possible routes, and every road has its own distance, so choosing the best route by hand is slow and easy to get wrong. Shortest-path algorithms solve this problem, but students often find them hard to understand because the steps happen inside the computer and cannot be seen.

This project, **Smart Route Finder**, is a desktop application written in Python with the Tkinter library. The road network is stored as a weighted undirected graph in which nodes are locations, edges are roads and weights are distances. The user can build a custom graph or load a ready-made example graph, choose a start node and a destination node, and then run **Dijkstra's algorithm**. The application also includes **Breadth First Search (BFS)** and **Depth First Search (DFS)** so that their results can be compared with Dijkstra's.

The search is animated step by step on a canvas and the final shortest path is highlighted together with its total distance. In the built-in example graph the shortest route from A to K is A → F → J → K with a total distance of 11. The implementation uses a min-heap (heapq) and runs in O((V + E) log V) time. The project makes a classic algorithm easy to see, test and understand.

Keywords: Shortest path, Dijkstra's algorithm, weighted graph, BFS, DFS, min-heap, Python, Tkinter, visualization.

Table of Content

Certificate ..... **I**
Acknowledgement ..... **II**
Abstract ..... **III**
Table of Content ..... **IV**
CHAPTER – 1 INTRODUCTION **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives of the Project 1
1.4 Scope of the Project 1
CHAPTER – 2 THEORETICAL BACKGROUND **2**
2.1 Graph and Basic Terms 2
2.2 Dijkstra's Algorithm 2
2.3 Breadth First Search (BFS) 3
2.4 Depth First Search (DFS) 3
2.5 Comparison of the Three Algorithms 3
CHAPTER – 3 SYSTEM DESIGN **4**
3.1 Architecture 4
3.2 Data Design 4
3.3 GUI Design 5
3.4 Algorithm Flowchart 6
CHAPTER – 4 IMPLEMENTATION **7**
4.1 Technologies Used 7
4.2 Main Methods of the Program 7
4.3 Important Code Explained 8
4.4 Animation and Error Handling 8
CHAPTER – 5 WORKED EXAMPLE **10**
CHAPTER – 6 RESULTS AND SCREENSHOTS **11**
6.1 Application Screenshots 11
6.2 Testing 14
6.3 Comparison of Algorithms 15
CHAPTER – 7 COMPLEXITY ANALYSIS **16**
CHAPTER – 8 ADVANTAGES, LIMITATIONS AND FUTURE SCOPE **17**
8.1 Advantages 17
8.2 Limitations 17
8.3 Future Scope 17
CHAPTER – 9 CONCLUSION **18**
REFERENCES **19**

CHAPTER – 1 INTRODUCTION

1.1 Background

Every day people travel from one place to another: from home to college, from one city to another, or from a warehouse to a shop. Between any two places there are usually many possible roads. Some are short and some are long. Map applications choose a good route in a fraction of a second using a **shortest path algorithm**.

In the subject Design and Analysis of Algorithms (DAA) we study how to design an algorithm, prove that it is correct and measure how fast it runs. The shortest path problem is one of the best examples for this study because it uses graphs, greedy decisions and a priority queue together. The most famous solution is **Dijkstra's algorithm**, published by Edsger W. Dijkstra in 1959.

Reading the steps of Dijkstra's algorithm in a book is not always easy, because we cannot see which node is picked, which distance is updated and why. This project, **Smart Route Finder**, was built to remove this difficulty. It lets the user draw a road network, run the algorithm and watch the search happen on the screen.

1.2 Problem Statement

Finding the shortest route between locations by hand is difficult for the following reasons:

1. Multiple routes can exist between two locations.
2. Every road has a different distance or cost.
3. We need the route with the minimum total distance, not just any route.
4. Shortest-path algorithms are hard to understand without visualization.

Solution: Smart Route Finder uses Dijkstra's algorithm to find and visualize the shortest path between two nodes of a weighted graph.

1.3 Objectives of the Project

The main objectives of this project are:

- To create a weighted graph of locations and roads.
- To let the user build a custom graph by adding and deleting nodes and edges.
- To assign a distance (weight) to every road.
- To select a Start Node and a Destination Node.
- To apply Dijkstra's algorithm and find the minimum-distance route.
- To animate the search process visually and highlight the final shortest path.
- To provide BFS and DFS so that their results can be compared with Dijkstra's.

1.4 Scope of the Project

Included in the project: a weighted undirected graph, an editable canvas, Dijkstra's algorithm, BFS and DFS, animation of the search, display of the final path and total distance, and messages for wrong or missing input.

Not included: real map data, GPS locations, one-way (directed) roads, negative distances, and live traffic information.

CHAPTER – 2 THEORETICAL BACKGROUND

2.1 Graph and Basic Terms

A **graph** is a set of points called nodes that are joined by lines called edges. In this project a road network is stored as a **weighted undirected graph**: weighted because every edge carries a number, and undirected because every road can be travelled in both directions.

Term	In graph theory	In Smart Route Finder
Node (vertex)	A point of the graph	A location or city, shown as a circle
Edge	A connection between two nodes	A road, shown as a grey line
Weight	A number on an edge	Distance or cost of the road
Start	Source node	The node where the journey begins (green)
Destination	Target node	The node where the journey ends (red)
Path	A sequence of connected nodes	A route such as A → F → J → K

Table 1: Graph terms used in the project

The program stores the graph as an **adjacency dictionary**. For every node it keeps a small dictionary of its neighbours and the weight of each road. For example, `edges["A"] = {"B": 4, "F": 3}` means that node A is connected to B with distance 4 and to F with distance 3. This form is simple, quick to look up and uses memory in proportion to the number of roads.

2.2 Dijkstra's Algorithm

Dijkstra's algorithm finds the shortest distance from one source node to all other nodes in a graph whose edge weights are not negative. It is a **greedy algorithm**: at every step it takes the unvisited node that is closest to the source, because no other route can reach that node with a smaller distance.

The main idea in simple words is: "Keep a distance tag on every node. Always work on the node with the smallest tag. If going through this node gives a neighbour a smaller distance, replace the neighbour's tag."

Steps of the algorithm

1. Set the distance of the start node to 0 and the distance of every other node to infinity.
2. Put the start node in a priority queue (min-heap).
3. Remove the node with the smallest distance from the queue.
4. For every neighbour, calculate new distance = distance of current node + weight of the road.
5. If the new distance is smaller than the neighbour's stored distance, update it, remember the current node as its previous node, and push it into the queue.
6. Repeat from step 3 until the destination is removed from the queue.
7. Follow the "previous" links backwards from the destination to rebuild the shortest path.

Why non-negative weights? Once a node is taken from the queue its distance is final. This is only true if no later road can make the distance smaller, and that is guaranteed only when all weights are zero or positive. The project therefore accepts only positive weights (minimum 0.1).

2.3 Breadth First Search (BFS)

BFS visits nodes level by level using a **queue**. It first visits all neighbours of the start node, then their neighbours, and so on. Because of this, BFS always finds the path with the **fewest edges**. However, it does not look at the weights, so the path with the fewest roads is not always the path with the smallest distance.

2.4 Depth First Search (DFS)

DFS goes as deep as possible along one road before coming back, using a **stack**. It is good for checking whether a route exists, but the path it finds depends on the order in which neighbours are explored. It gives no promise of a short path, as the results in Chapter 6 show.

2.5 Comparison of the Three Algorithms

Property	Dijkstra	BFS	DFS
Data structure	Min-heap (heapq)	Queue (deque)	Stack (list)
Uses edge weights	Yes	No	No
Shortest by distance	Always (non-negative weights)	No (fewest edges only)	No
Time complexity	O((V + E) log V)	O(V + E)	O(V + E)
Best used for	Shortest distance routes	Fewest-stops routes	Checking if a route exists

Table 2: Dijkstra, BFS and DFS compared

CHAPTER – 3 SYSTEM DESIGN

3.1 Architecture

The application is a single Python class, `SmartRouteFinder`, with three working parts: the GUI layer, the graph data and the algorithms. The user clicks on the canvas; the GUI changes the graph data; when the user presses **Find Best Path** the chosen algorithm reads the graph and returns the visited order, the final path and (for Dijkstra) the distances. The animation part then draws these results on the canvas one step at a time.

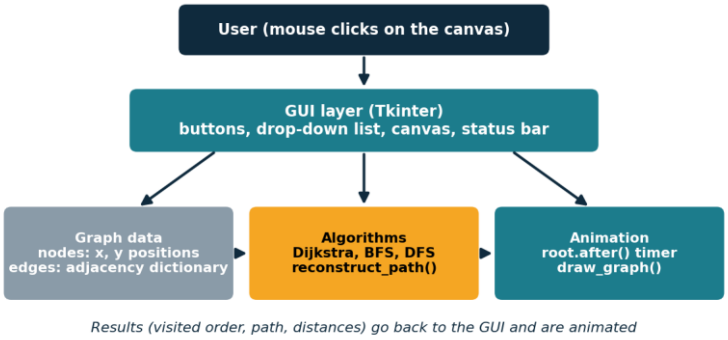


Figure 1: Architecture of Smart Route Finder

3.2 Data Design

The main data items of the program are listed below.

Variable	Purpose
`nodes`	Dictionary of node name → position, for example {"A": {"x": 100, "y": 100}}
`edges`	Adjacency dictionary of node → {neighbour: weight}
`start_node`, `end_node`	The selected source and destination
`selected_node`	First node clicked while adding or deleting an edge (drawn orange)
`mode`	Current tool: add_node, add_edge, delete_node, delete_edge, set_start, set_end
`running`	True while the animation is playing; blocks clicks and new searches

Table 3: Main data items

The built-in example graph has 11 nodes (A to K) and 17 roads.

Road	Dist.	Road	Dist.	Road	Dist.
A–B	4	D–E	5	G–J	4
A–F	3	D–H	2	H–I	2
B–C	2	E–I	3	H–K	5
B–F	5	F–G	2	I–K	3
C–D	3	F–J	6	J–K	2
C–G	4	G–H	3		

Table 4: Roads of the example graph

3.3 GUI Design

The window has a title, a control panel, a row of graph tools, a drawing canvas, a status line and a colour legend.

Control	What it does
Algorithm drop-down	Chooses Dijkstra, BFS or DFS
Find Best Path	Runs the chosen algorithm and starts the animation
Example Graph / Reset	Loads the 11-node example graph with A as start and K as destination
Clear Graph	Deletes all nodes and edges after asking for confirmation
Add Node	Click on empty canvas space to create the next node (A, B, C …)
Add Edge	Click two nodes, then type the distance in a dialog box
Delete Node / Delete Edge	Remove a node with all its roads, or the road between two clicked nodes
Set Start / Set Destination	Click a node to make it the start (green) or destination (red)

Table 5: Controls of the application

Colour on screen	Meaning
Green circle	Start node
Red circle	Destination node
Light blue circle	Normal node
Orange circle	Node selected while adding or deleting an edge
Sky blue circle	Node being processed by the search at this moment
Yellow circle and thick orange line	Final shortest path

Table 6: Colour legend

3.4 Algorithm Flowchart

Figure 2 shows how Dijkstra's algorithm is implemented in the project.

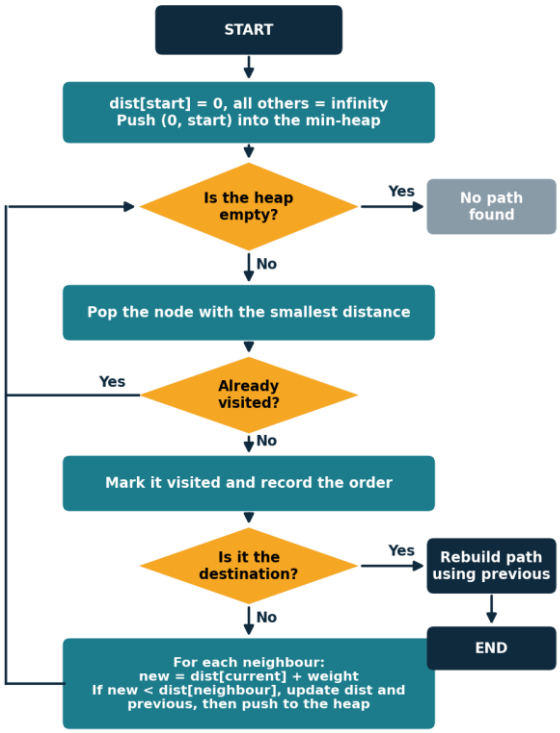


Figure 2: Flowchart of Dijkstra's algorithm as implemented

CHAPTER – 4 IMPLEMENTATION

4.1 Technologies Used

Technology	Use in the project
Python 3	Programming language
Tkinter	Window, buttons, drop-down list, dialog boxes and the drawing canvas
Dictionary	Storage of nodes and the adjacency list of roads
heapq	Min-heap used as the priority queue in Dijkstra's algorithm
collections.deque	Queue used in BFS
math	Distance between two points, used to find which node was clicked

Table 7: Technology stack

4.2 Main Methods of the Program

Method	Work done
`create_gui()`	Builds all widgets of the window
`set_mode()`, `canvas_click()`	Select a tool and react to mouse clicks according to the tool
`add_edge()`, `delete_node()`, `delete_edge()`	Change the graph; roads are stored in both directions
`get_node_at()`	Finds the node under the mouse using the distance formula
`draw_graph()`	Clears the canvas and redraws roads, weights and nodes
`dijkstra()`, `bfs()`, `dfs()`	Return (visited order, path, distances)
`reconstruct_path()`	Builds the path from the "previous" dictionary
`find_path()`	Checks input, runs the selected algorithm, starts animation
`animate_visited()`, `animate_final_path()`	Show the search and then the final path using timers
`create_example_graph()`, `clear_graph()`	Load the sample graph or empty the graph

Table 8: Important methods of class SmartRouteFinder

4.3 Important Code Explained

Dijkstra's algorithm (shortened from the project code)

```
distances = {node: float("inf") for node in self.nodes}
previous = {}
distances[self.start_node] = 0
priority_queue = [(0, self.start_node)]
visited, visited_order = set(), []

while priority_queue:
    current_distance, current = heapq.heappop(priority_queue)
    if current in visited:
        continue
    visited.add(current)
    visited_order.append(current)
    if current == self.end_node:
        break
    for neighbor, weight in self.edges[current].items():
        new_distance = current_distance + weight
        if new_distance < distances[neighbor]:
            distances[neighbor] = new_distance
            previous[neighbor] = current
            heapq.heappush(priority_queue, (new_distance, neighbor))
```

`heappop` always returns the pair with the smallest distance, so the greedy choice is made in O(log V) time. A node may be pushed more than once when a better distance is found later; the line `if current in visited: continue` simply ignores the older, longer entries. The loop stops as soon as the destination is taken out of the heap, so the program does not waste time on nodes that are farther away.

Rebuilding the path

```
path = []
current = self.end_node
while current != self.start_node:
    path.append(current)
    current = previous[current]
path.append(self.start_node)
path.reverse()
```

Every time a distance is improved, the program remembers from which node it came in `previous`. Starting at the destination and following these links back to the start gives the path in reverse order, so the list is reversed at the end. If the destination never receives a previous node, the function returns an empty list and the program shows a "No Path" message.

4.4 Animation and Error Handling

The animation uses the Tkinter timer `root.after()`. `animate_visited()` takes one node from the visited list every 350 milliseconds, redraws the graph and marks the current node in sky blue. When all nodes are shown, `animate_final_path()` draws the shortest path node by node every 400 milliseconds with yellow circles and thick orange roads. While this runs, the flag `running` is True, so the user cannot click or start another search. The status line finally shows the path and the total distance.

Situation	Behaviour of the program
New node placed closer than 50 pixels to another node	Not created; status line says the node is too close
Edge from a node to itself	Rejected with a message
Edge that already exists	Information box “Edge Exists”
Edge weight smaller than 0.1 or not a number	Dialog box does not accept it
Find Best Path with no graph, start or destination	Warning box asks the user to create the graph or select the node
No route between start and destination	Information box “No Path”
Delete Node or Clear Graph	Asks for confirmation first

Table 9: Input checks in the application

CHAPTER – 5 WORKED EXAMPLE

To understand how the algorithm works, let us follow Dijkstra's algorithm on the example graph from the start node A to the destination K. Table 10 is produced by running the same logic as the project code. In each row the node with the smallest distance is taken from the heap and the distances of its neighbours are checked.

Step	Node taken	Distance	Distances updated (new value)
1	A	0	B=4, F=3
2	F	3	G=5, J=9
3	B	4	C=6
4	G	5	H=8
5	C	6	D=9
6	H	8	I=10, K=13
7	D	9	E=14
8	J	9	K=11
9	I	10	E=13
10	K	11	destination reached, stop

Table 10: Step-by-step run of Dijkstra's algorithm from A to K

When K is taken from the heap with distance 11, the destination is reached and the search stops. Following the previous nodes back gives K ← J ← F ← A, so the shortest path is **A → F → J → K** with total distance **3 + 6 + 2 = 11**. Notice that K was first reached from H with distance 13, but a shorter distance of 11 was found later through J and replaced it. This is exactly the "update when a shorter route is found" rule of the algorithm.

We can also check the answer by listing the main routes from A to K:

No.	Route	Calculation	Total
R1	A → F → J → K	3 + 6 + 2	11
R2	A → F → G → J → K	3 + 2 + 4 + 2	11
R3	A → F → G → H → K	3 + 2 + 3 + 5	13
R4	A → F → G → H → I → K	3 + 2 + 3 + 2 + 3	13
R5	A → B → C → D → H → K	4 + 2 + 3 + 2 + 5	16

Table 11: Possible routes from A to K (R1 and R2 tie for the shortest)

Two routes, R1 and R2, have the same shortest distance of 11. The application shows A → F → J → K because it reaches K first through J.

CHAPTER – 6 RESULTS AND SCREENSHOTS

6.1 Application Screenshots

The following screenshots show the Smart Route Finder window in different situations.

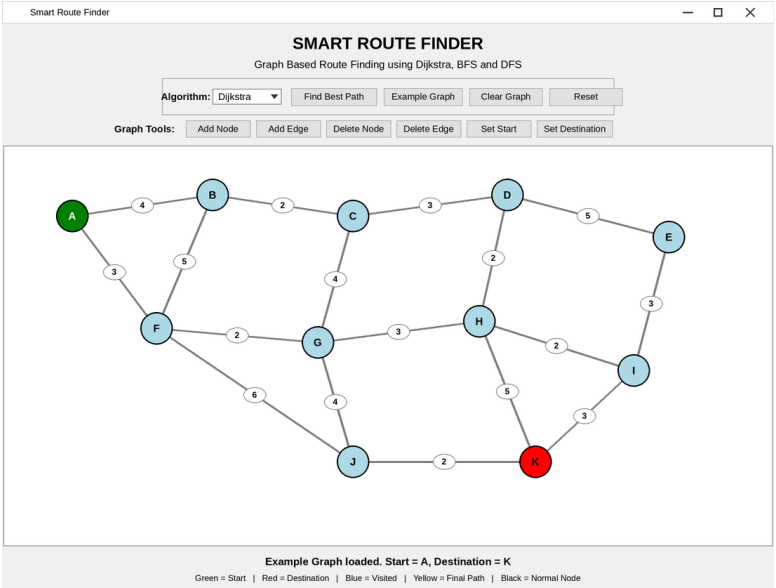


Figure 3: Main window with the example graph loaded (Start = A, Destination = K)

Figure 3 shows the main window after the example graph is loaded. A is green (start) and K is red (destination). The grey numbers on the roads are distances.

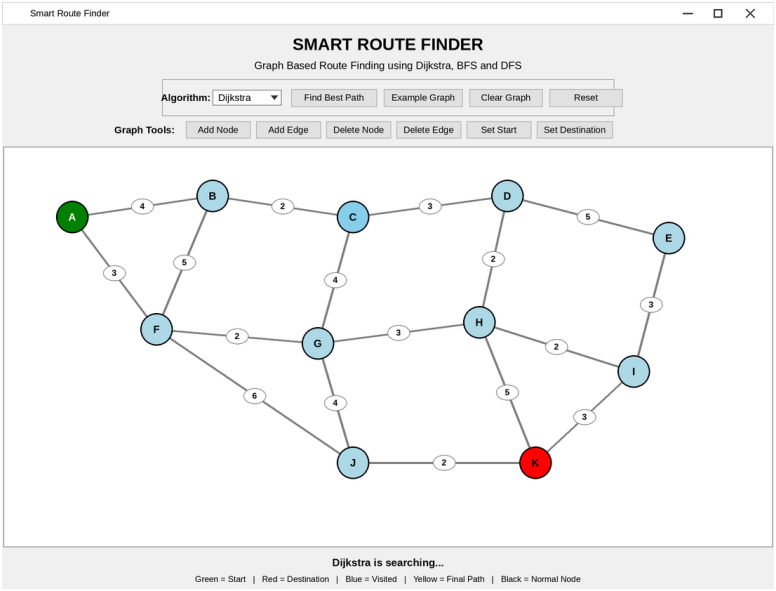


Figure 4: Dijkstra searching – the node being processed is shown in sky blue

Figure 4 shows the search in progress. The status line says “Dijkstra is searching…” and the buttons are locked until the animation ends.

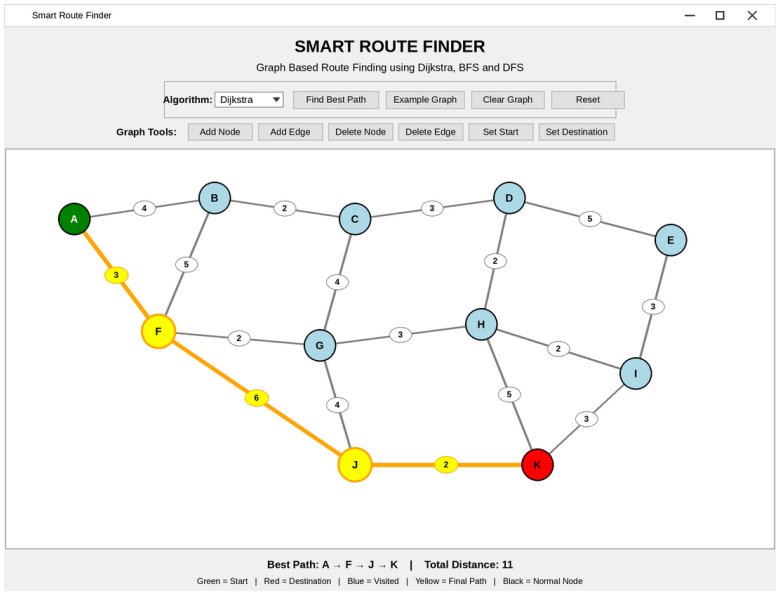


Figure 5: Final result of Dijkstra: A → F → J → K with total distance 11

Figure 5 shows the final result. The shortest path is drawn with yellow circles and thick orange roads, and the status line gives the path and the total distance 11.

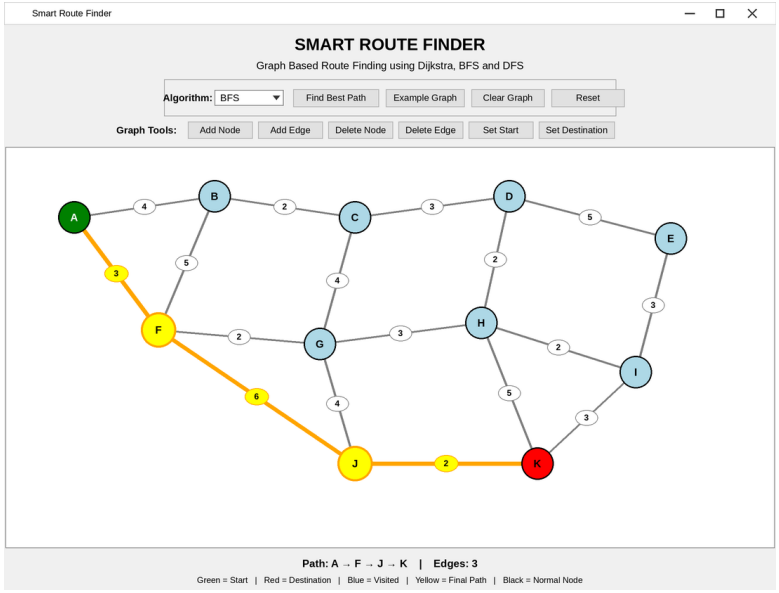


Figure 6: Result of BFS for the same graph (fewest edges)

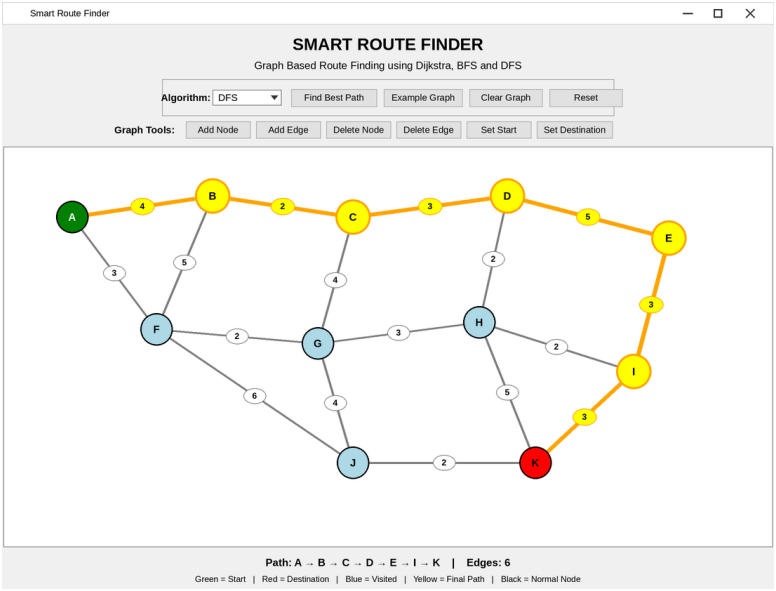


Figure 7: Result of DFS for the same graph

Figures 6 and 7 show BFS and DFS on the same graph. BFS finds a path with 3 roads (distance 11, the same as Dijkstra by chance). DFS goes deep first and returns the long route A → B → C → D → E → I → K with 6 roads and distance 20, which is almost twice the shortest distance.

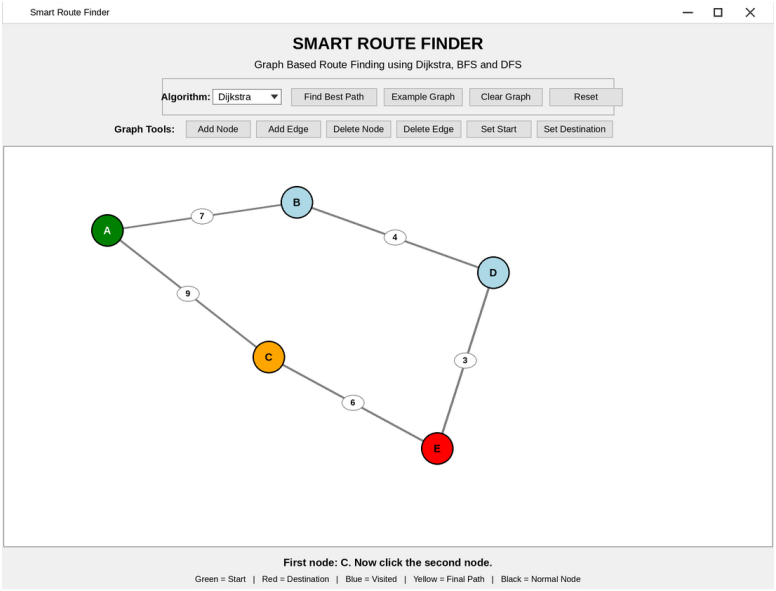


Figure 8: Building a custom graph – Add Edge tool with node C selected (orange)

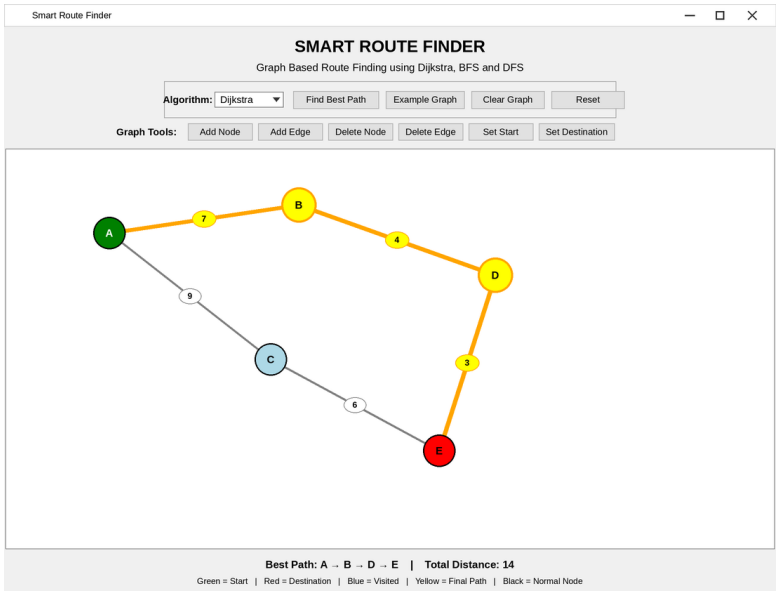


Figure 9: Dijkstra on the custom graph: A → B → D → E with total distance 14

Figures 8 and 9 show a user-made graph with five nodes. In Figure 8 the user is adding an edge and the first node C is shown in orange. In Figure 9 Dijkstra chooses A → B → D → E (7 + 4 + 3 = 14), which is shorter than the other route A → C → E (9 + 6 = 15).

6.2 Testing

The algorithms of the project were run on several test cases. All results below were produced by the project's own `dijkstra()`, `bfs()` and `dfs()` methods.

No.	Test case	Dijkstra	BFS	DFS
1	Example graph, A to K	A→F→J→K (11)	A→F→J→K (11)	A→B→C→D→E→I→K (20)
2	Example graph, B to I	B→C→D→H→I (9)	B→C→D→E→I (13)	B→C→D→E→I (13)
3	Example graph, A to E	A→F→G→H→I→E (13)	A→B→C→D→E (14)	A→B→C→D→E (14)
4	Same start and destination (A to A)	A (0)	A (0)	A (0)
5	Custom graph, A to E	A→B→D→E (14)	A→C→E (15)	A→B→D→E (14)
6	Disconnected node (A to F)	No path	No path	No path

Table 12: Test cases and results (total distance in brackets)

The tests confirm that Dijkstra gives the smallest total distance in every case where a route exists. When the start and the destination are the same node, every algorithm returns that single node with distance 0. When a node is not connected to the rest of the graph, all algorithms return an empty path and the program shows the "No Path" message.

6.3 Comparison of Algorithms

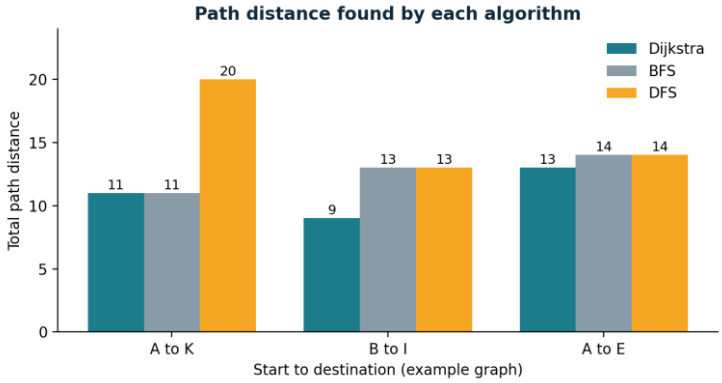


Figure 10: Total distance found by Dijkstra, BFS and DFS on the example graph

Figure 10 compares the three algorithms on three different start and destination pairs. Dijkstra is always the lowest. For example, from B to I, BFS and DFS return distance 13 while Dijkstra returns 9; from A to E they return 14 while Dijkstra returns 13. This shows clearly that **an algorithm which ignores weights cannot be trusted to give the shortest distance**.

CHAPTER – 7 COMPLEXITY ANALYSIS

Let V be the number of nodes (locations) and E the number of edges (roads).

Measure	Result	Reason
Time complexity of Dijkstra	O((V + E) log V)	Every edge can cause one heap push and every heap operation costs O(log V)
Space complexity	O(V + E)	Adjacency dictionary plus the distance, previous and visited structures
BFS and DFS time	O(V + E)	Every node and edge is looked at at most once
Finding a clicked node	O(V)	All nodes are checked with the distance formula

Table 13: Complexity of the project

A simple version of Dijkstra's algorithm that searches for the smallest distance in a plain list takes $O(V^2)$ time. Using a min-heap (the `heapq` module) makes it much faster for graphs with few roads per node, which is the normal case for road networks. In the project, the heap holds pairs of (distance, node) and the algorithm stops early when the destination is reached.

To check the speed in practice, the project's `dijkstra()` method was timed on random connected graphs with about three roads per node. The best of five runs is shown in Figure 11.

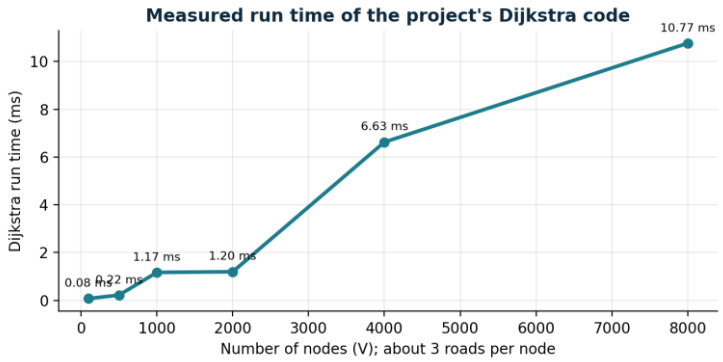


Figure 11: Measured run time of Dijkstra's algorithm for growing graph sizes

Even for a graph with 8000 nodes the search finishes in about ten milliseconds, so the algorithm itself is never the slow part of the application. Most of the waiting time the user sees comes from the animation delays (350 ms and 400 ms per step), which are added on purpose so that every step can be seen. The timings were measured on the machine used to prepare this report and will differ on other computers; the growth trend is what matters.

CHAPTER – 8 ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

8.1 Advantages

- The algorithm is **easy to see**: the search and the final path are animated.
- The user can **build any graph**, not only the example one.
- Dijkstra's result is **always the minimum distance** for non-negative weights.
- BFS and DFS are included, so students can **compare** algorithms on the same graph.
- Written in plain Python with standard libraries only, so it **needs no installation** of extra packages.
- Input checks and confirmation boxes protect the user from mistakes.

8.2 Limitations

- Only non-negative weights are supported (Dijkstra's algorithm cannot handle negative roads).
- All roads are two-way; one-way roads are not available.
- When two routes have the same distance, only one of them is shown.
- The window has a fixed size, and the canvas cannot be zoomed or scrolled.
- Locations are drawn by hand on the canvas; real map data is not used.
- The graph cannot be saved to a file and loaded again.

8.3 Future Scope

- Real-world map integration and GPS-based locations.
- Real-time traffic information and dynamic road distances.
- Comparison with the A* algorithm using a distance estimate (heuristic).
- Real-time route optimization.
- Support for one-way roads, saving and loading graphs, and showing the distance table of every node during the search.

CHAPTER – 9 CONCLUSION

The Smart Route Finder project shows how Dijkstra's algorithm finds the shortest route in a weighted graph and turns a difficult topic into something that can be seen on the screen. The user can create nodes and roads, choose the start and destination, run Dijkstra, BFS or DFS, and watch the search and the final route. In the example graph the shortest route from A to K is A → F → J → K with a total distance of 11, and the tests showed that Dijkstra always gave the smallest distance while BFS and DFS sometimes gave routes that were longer.

During the project I learned how to represent a graph with an adjacency dictionary, how a priority queue makes a greedy algorithm efficient, how the complexity O((V + E) log V) comes from heap operations, and how to build an interactive Tkinter application with animation. The project can be extended further with real maps, traffic data and the A* algorithm, which makes it a good base for future work.

REFERENCES

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C., Introduction to Algorithms, MIT Press (chapters on graph algorithms and shortest paths).
2. Levitin, A., Introduction to the Design and Analysis of Algorithms, Pearson.
3. Dijkstra, E. W., "A note on two problems in connexion with graphs", Numerische Mathematik, Vol. 1, 1959, pp. 269–271.
4. Python Software Foundation, heapq – Heap queue algorithm, https://docs.python.org/3/library/heapq.html
5. Python Software Foundation, collections – deque objects, https://docs.python.org/3/library/collections.html
6. Python Software Foundation, tkinter – Python interface to Tcl/Tk, https://docs.python.org/3/library/tkinter.html
7. Course material of Design and Analysis of Algorithms, A D Patel Institute of Technology.